The Constrained LASSO is a Portfolio

A numerical check of the CLA–LASSO identity under constraints

Thomas Schmelzer
Jebel Quant Research, Abu Dhabi
ORCID 0009-0009-9771-3501

24 October 2026

Jebel-Quant/lasso 5395f07
24 Oct 2026

Abstract

Schmelzer and Hastie showed that the solution path of the constrained LASSO and the path of Markowitz's Critical Line Algorithm (CLA) are the same curve. This note checks that identity numerically. It poses a constrained regression as a portfolio, in a penalty form or a budget form, and hands it to a released CLA written for portfolios; where an encoding starts on a degenerate vertex, the CLA runs twice on a relaxed problem and the two paths are combined and certified. On caps, signs, equality and inequality rows, monotone fits and zero sums, on simulated and real data, up to 500 predictors and on problems built to break a path algorithm, the paths reproduce LARS, an interior-point solver and the zero-sum path algorithm comlasso, and meet the LASSO's optimality conditions to rounding. One problem, a design with two nearly identical columns, is declined with an error rather than answered wrongly. With more predictors than observations the paths stay exact under bounds, signs, inequality rows and zero sums. The path algorithms for general constraints add a ridge there, and the ridge moves the end of the path.

Keywords: constrained LASSO; critical line algorithm; solution path; compositional data; portfolio optimisation; reproducibility

1 A constrained regression

Tibshirani [21] introduced the LASSO as least squares with a single constraint, a budget on the ℓ_1 norm of the coefficients:

$$\min_{\beta} \ \tfrac{1}{2}\|y - X\beta\|_2^2 \quad \text{s.t.} \quad \|\beta\|_1 \le t, \qquad \text{or equivalently} \qquad \min_{\beta} \ \tfrac{1}{2}\|y - X\beta\|_2^2 + \lambda\|\beta\|_1.$$

LARS [4] and the homotopy method [13] compute the solution for every t, or every λ, at once. The solution is piecewise linear in the parameter, so the whole *path* is a finite list of *knots*. scikit-learn's `lars_path` [15] takes X, y and one option, `positive`, for nonnegative coefficients. There is no place for any other constraint.

Many regressions need one. The diabetes data that LARS was introduced with [4] are an example: $n = 442$ patients, ten standardised baseline variables and, as the response, a measure of disease progression a year later. Six of the variables are blood serum measurements, and three of them are closely related measures of cholesterol: total cholesterol (`s1`), LDL (`s2`) and HDL (`s3`). Least squares plays them off against each other. It gives total cholesterol a coefficient of -792 and LDL one of 477, two large effects of opposite sign that mostly cancel, and it gives HDL, whose low levels are a risk factor for diabetes [24], a coefficient of 101, as if more of it made the disease progress faster. The LASSO path ends at the least-squares fit, so it gets there too. A cap on every coefficient, $|\beta_j| \le u$, stops the offsetting; we take $u = 396.1$, half the largest least-squares coefficient.

Figure 1 shows what the cap does. Without it the path passes the cap at ten of its 13 knots and ends with HDL positive. With it no coefficient passes the cap, and HDL stays negative along the whole path, although no sign is imposed. That does not hang on the choice of u: every whole-number cap up to 637 keeps HDL negative along the whole path, and only looser caps let it turn positive. The capped path has 13 knots.

Both panels are exact paths. They come from the Critical Line Algorithm (CLA), which Markowitz [11] wrote for portfolios, applied to one of the two encodings described below. The open-source package `cvxcla` [18] carries them out (Algorithm 1), and the right panel is one call:

```
from cvxcla import Lasso
knots = Lasso(X, y, lower=-u * np.ones(p), upper=u * np.ones(p)).path
```

The same call takes signs (a bound of zero), equality rows `a, b` and inequality rows `g, h`, in combination, and picks the encoding that takes them.

What this note shows Schmelzer and Hastie [17] prove that the CLA and the constrained LASSO [7, 5] trace the same curve. Table 1 gives the dictionary: the coefficients are positions, $X^\top X$ is the covariance of the assets, $X^\top y$ is their expected return and λ is a risk tolerance. Portfolio managers have always needed constraints, such as a budget, position limits and sector exposures, and Markowitz built them into the CLA from the start. So a CLA written for portfolios, with a thin layer that poses the regression as a portfolio, should give exact constrained-LASSO paths. This note shows that it does, and checks every path against references that know nothing of portfolios. The layer is two encodings, Propositions 1 and 2, proved here from the definition of the CLA, and, where an encoding starts degenerate, a relaxation that runs the CLA twice and combines the two paths (Section 6). The checks also exposed two defects in the CLA that refused ordinary portfolio problems too; they were fixed in `cvxcla` 2.6.0 (Appendix D), and the release used here is 2.7.0. In one case that matters in practice the result goes beyond the other open path algorithms for general constraints: with more predictors than observations it traces the exact path, with no ridge (below). The two propositions need nothing from [17] beyond the classical equivalence of the bound and penalty forms of the LASSO; that paper supplies the general identity of which they are instances. Brodie et al. [2] put an ℓ_1 penalty on Markowitz portfolios to make them sparse, and a companion note [19] computes efficient frontiers with a LASSO solver. Both use regression to build portfolios. This note goes the other way.

On their own the encodings need a nondegenerate start; where they do not have one, `cvxcla` relaxes the constraint (Section 6). There are two, and both are needed:

1. the *penalty form* (Section 3) keeps λ as the parameter, and takes coefficient bounds, general equality constraints $A\beta = b$ and inequality rows $C\beta \le h$. It is the general form, and the one that solves the example;
2. the *budget form* (Section 4) keeps the ℓ_1 budget, as Markowitz's long-only, fully invested problem with a cash asset. It takes homogeneous constraints such as the zero sum $\mathbf{1}^\top\beta = 0$ of a regression on compositional data directly, in one run. The penalty form reaches a zero sum only by relaxing it, at the cost of a second run (Appendix B).

Each is a few lines on top of a released CLA. Readers who know the CLA can skip Section 2.

Related work Exact paths of the constrained LASSO are not new. Rosset and Zhu [16] characterise the regularised problems whose solution path is piecewise linear, a class that contains it. Zhou and Lange [25] trace the solution of a quadratic program under linear equality and inequality constraints as an exact penalty on the constraints grows. The generalised lasso of Tibshirani and Taylor [22], with penalty $\lambda\|D\beta\|_1$, has an exact path through its dual. Gaines, Kim and Zhou [5] derive a path algorithm for the constrained LASSO itself from its optimality conditions. It tracks the active set and the multipliers of the linear constraints as λ decreases,

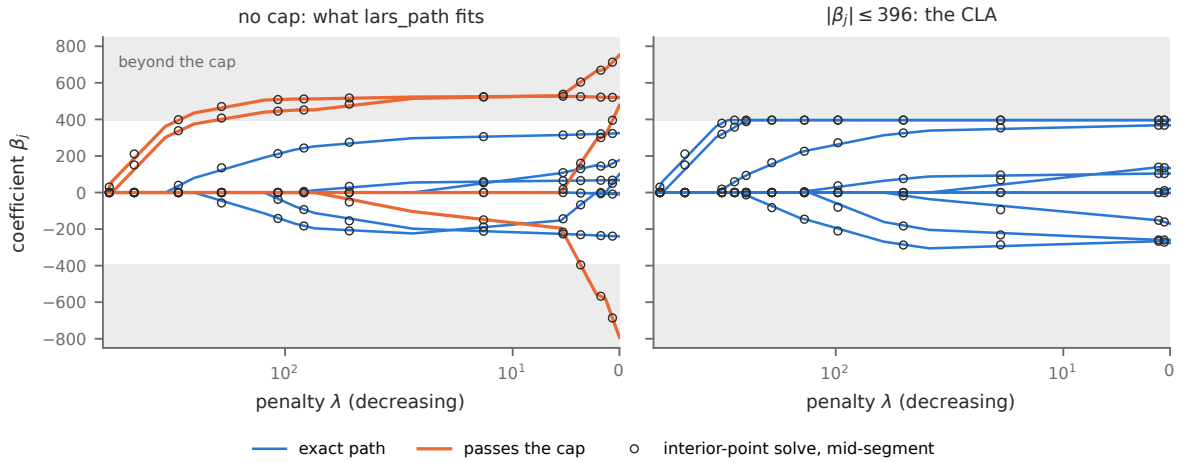


Figure 1: The opening example: the diabetes LASSO path without and with the cap $|\beta_j| \le u = 396.1$. The shaded bands are beyond the cap. Left: the plain LASSO, the path `lars_path` computes; the coefficients that pass the cap are in orange. Right: every coefficient capped. Lines: the exact path, by the CLA in the penalty form. Circles: an interior-point solve at the midpoint of every segment.

regression	portfolio
coefficient β_j	position in asset j
$X^\top X$	covariance of the asset returns
$X^\top y$	expected returns (penalty form: a benchmark)
$\|\beta\|_1$	gross exposure
penalty λ	risk tolerance γ (penalty form)
bound $\|\beta\|_1 \le t$	fully invested budget, $\gamma = 1/t$ (budget form)
$A\beta = b$, $l \le \beta \le u$	budget rows and position limits
knot of the LASSO path	turning point of the CLA

Table 1: The dictionary between the two problems [17].

and it takes general linear rows. The two encodings alone start degenerate on some of them, homogeneous inequality rows and caps or rows under a zero sum; `cvxcla` reaches those by relaxing the constraint (Section 6). For $C\beta = 0$ alone, the Python package `c-lasso` [20] has a path algorithm with a small ridge term added. Neither was derived with portfolios in mind, so Section 5 uses both as independent checks. For zero sums on blocks of coefficients, the comlasso algorithm of Jeon et al. [8] traces the exact path, also with $p > n$; it takes no bounds or inequality rows, and Section 5 runs it too. Hobson, Butcher and Lentati [6] revisit Gaines et al. with an algorithm of their own, CLASSO. The linear system of Gaines et al. is singular whenever a binding row vanishes on the active coefficients, as a sign constraint on a zero coefficient does, so CLASSO takes the direction from a small quadratic program instead. They also show that a path with finitely many knots need not be traversed in finitely many steps: recomputing the multipliers at every knot can step forever along one segment. Following one canonical certificate terminates, and they check the proofs in Lean. CLASSO adds a ridge when $n < p$, and its code had not been released when we wrote, so we do not run it. The CLA is an active-set method of the same kind, following one family of quadratic programs; the difference is that it was written for portfolios and is used here as released.

More predictors than observations With $p > n$ the encodings part from the other path algorithms for general constraints. Gaines et al., CLASSO and `c-lasso` add a ridge to $X^\top X$

there, and so trace a nearby problem. With $p > n$ that problem is not close at its end: the ridge decides which interpolating fit the path runs to. As shipped, `c-lasso` ends up to 0.58 times the largest coefficient away from the exact path on a zero-sum problem with $p > n$ (Section 5). The CLA needs no ridge, and traces the exact path under bounds, signs, inequality rows and zero sums alike (Table 5). Microbiome studies often have more taxa than samples, so this is the case a user of the zero-sum LASSO meets first.

Notation The design is $X \in \mathbb{R}^{n\times p}$, with n observations and p predictors, and the response is $y \in \mathbb{R}^n$. Write $G = X^\top X$ and $c = X^\top y$. The constrained LASSO is

$$\mathrm{L}_\lambda : \quad \min_\beta \ \tfrac{1}{2}\|y - X\beta\|_2^2 + \lambda\|\beta\|_1 \quad \text{s.t.} \quad A\beta = b, \quad C\beta \le h, \quad l \le \beta \le u, \tag{1}$$

with $A \in \mathbb{R}^{m\times p}$, $C \in \mathbb{R}^{k\times p}$ and $l_j \le 0 \le u_j$, infinite entries allowed, and a nonempty feasible set. A bound that excludes zero, such as $\beta_j \ge \ell$ with $\ell > 0$, is written as the row $-\beta_j \le -\ell$. With no rows and infinite bounds it is Tibshirani's LASSO. We assume, as [17] does, that L_λ has a unique minimiser $\beta(\lambda)$ for every $\lambda > 0$. For the LASSO without constraints this holds whenever the columns of X are in general position [23], for any n and p. With constraints, and in particular with $p > n$, we know of no such result and make it an assumption. The checks of Section 5 test optimality, not uniqueness: where the minimiser is not unique they confirm that the path is a path of minimisers.

2 The Critical Line Algorithm

In words, the CLA starts with the riskiest portfolio, the one with the highest expected return, and lowers the risk tolerance step by step. Between steps the optimal portfolio moves along a straight line. A step ends when a position hits a limit or an asset at a limit wants to come off it. These corners are the *turning points*; the LASSO calls them knots.

Precisely, the CLA solves the family of quadratic programs

$$\mathrm{P}_\gamma : \quad \min_x \ \tfrac{1}{2}\, x^\top \Sigma x - (q + \gamma\mu)^\top x \quad \text{s.t.} \quad Ax = b, \quad Cx \le h, \quad l \le x \le u, \tag{2}$$

for every γ from $+\infty$ down to 0, with $\Sigma \succeq 0$. Markowitz's algorithm has the equality rows and the bounds; the inequality rows are a standard extension, handled like the bounds. Here x is the vector of holdings, Σ the covariance, μ the expected returns and γ the risk tolerance. The term q is a benchmark: with $q = \Sigma\bar{x}$ the objective is, up to a constant, the tracking variance against a portfolio $\bar{x}$ less $\gamma\mu^\top x$, which is the same CLA after the shift $x - \bar{x}$ of the bounds and the right-hand side. With $q = 0$ it is Markowitz's problem.

Segments Write $g = \Sigma x - q - \gamma\mu + A^\top\nu + C^\top\eta$ for the gradient of the Lagrangian. A point x with multipliers ν and η is optimal when $g_j = 0$ on the *free* coordinates $\mathcal{F} = \{j : l_j < x_j < u_j\}$, $g_j \ge 0$ where $x_j = l_j$, and $g_j \le 0$ where $x_j = u_j$; and when $\eta \ge 0$, with $\eta_i = 0$ on every row that is slack, $C_i x < h_i$. Call the coordinates at a bound *blocked*, $\mathcal{B}$, and the rows held tight *active*, $\mathcal{R}$. An active row is held as an equality: in the system below it is stacked under A, its right-hand side h_i under b and its multiplier η_i under ν, and the slack rows take no part. To keep the display short, A, b and ν stand for these stacks. With $\mathcal{F}$, $\mathcal{B}$ and $\mathcal{R}$ fixed, $x_\mathcal{F}$ and ν solve the bordered system

$$\begin{bmatrix} \Sigma_{\mathcal{F}\mathcal{F}} & A_\mathcal{F}^\top \\ A_\mathcal{F} & 0 \end{bmatrix} \begin{bmatrix} x_\mathcal{F} \\ \nu \end{bmatrix} = \begin{bmatrix} q_\mathcal{F} - \Sigma_{\mathcal{F}\mathcal{B}} x_\mathcal{B} \\ b - A_\mathcal{B} x_\mathcal{B} \end{bmatrix} + \gamma \begin{bmatrix} \mu_\mathcal{F} \\ 0 \end{bmatrix}, \tag{3}$$

so $x_\mathcal{F} = \alpha + \gamma\delta$ and $\nu = \nu_0 + \gamma\nu_1$ are affine in γ. The gradient on the blocked coordinates is affine too, $g_\mathcal{B} = g_0 + \gamma g_1$. One factorisation and two right-hand sides give the whole segment.

Events Lowering γ, the segment ends at the largest γ below the current one at which either a free coordinate reaches a bound, $\gamma = (l_i - \alpha_i)/\delta_i$ for $\delta_i > 0$ or $(u_i - \alpha_i)/\delta_i$ for $\delta_i < 0$, or a blocked coordinate's gradient changes sign, $\gamma = -g_{0j}/g_{1j}$ for $g_{1j} > 0$ at a lower bound or $g_{1j} < 0$ at an upper one. The rows have the same two events: a slack row's slack $h_i - C_i x$, affine in γ, reaches zero, and the row joins $\mathcal{R}$; or an active row's multiplier η_i, affine like ν, falls to zero, and the row leaves it. The coordinate or row concerned changes sides, (3) is solved again, and the walk continues. If there is no event the last segment runs to $\gamma = 0$.

The start As $\gamma \to \infty$ the linear term dominates, and the path starts at a vertex maximising $\mu^\top x$ over the feasible set. A linear program finds it. If the maximiser is unique and has as many coordinates strictly inside their bounds as there are equality rows and rows tight at it, the bordered matrix in (3) is nonsingular and the walk can begin. This is the nondegenerate case. `cvxcla` declines a degenerate start.

3 The penalty form

The first encoding reads the penalty $\lambda\|\beta\|_1$ as a cost: every unit of gross position, long or short, costs λ units of return. To make that a linear cost, each coefficient is split into a long and a short position, both nonnegative. The constraints on β become constraints on the positions, unchanged in kind.

Formally, let S have a column e_j for every j with $u_j > 0$ and a column $-e_j$ for every j with $l_j < 0$, so $\beta = Sz$ with $z \geq 0$, and let $\bar{u}$ collect the matching bounds u_j and $-l_j$. A coefficient whose bounds are both zero is fixed and can be dropped, so every j has at least one column and S has full row rank.

Proposition 1 (Penalty as cost)**.** *For $\lambda > 0$, $\beta(\lambda) = Sz(\lambda)$, where $z(\lambda)$ solves the CLA instance P_γ with*

$$\Sigma = S^\top G S, \quad \mu = -\mathbf{1}, \quad q = S^\top c, \quad \text{rows } ASz = b,\ CSz \leq h, \quad 0 \leq z \leq \bar{u}, \quad \gamma = \lambda.$$

Moreover q lies in the range of Σ, so the benchmark reading of Section 2 applies: the program tracks a least-squares fit and pays λ for every unit of gross position.

Proof. Up to the constant $\frac{1}{2}\|y\|^2$, the CLA objective at $\gamma = \lambda$ is $\frac{1}{2}\|y - XSz\|^2 + \lambda \mathbf{1}^\top z$. For a fixed β feasible in (1), the smallest $\mathbf{1}^\top z$ over $z \geq 0$ with $Sz = \beta$ and $z \leq \bar{u}$ is $\|\beta\|_1$; the rows hold for every such z, since they constrain $Sz = \beta$ only. It is attained by putting β_j on the column of its sign, which respects $\bar{u}$ because $l_j \leq \beta_j \leq u_j$. The minimum of the CLA program over z is therefore the minimum of L_λ over β. At a minimiser no coordinate holds both signs, since lowering both by the same amount keeps β and reduces the cost by 2λ times that amount. Hence $Sz(\lambda)$ minimises L_λ, and by uniqueness equals $\beta(\lambda)$. For the range claim, $c = X^\top y$ lies in the range of G, say $c = Gv$. Since S has full row rank, $v = S\bar{x}$ for some $\bar{x}$, and then $q = S^\top G S\bar{x} = \Sigma\bar{x}$, where $S\bar{x}$ is a least-squares fit. □

In code, on top of `cvxcla`'s `CLA`, which implements Section 2, and with both signs allowed for every coefficient:

```
S = np.hstack([np.eye(p), -np.eye(p)])        # beta = S z with z >= 0
run = CLA(mean=-np.ones(2 * p), covariance=S.T @ G @ S, a=A @ S, b=b, g=C @ S, h=h,
          lower_bounds=np.zeros(2 * p), upper_bounds=cap, q=S.T @ c)
knots = [(t.lamb, S @ t.weights) for t in run.turning_points]   # lam = gamma
```

The covariance $S^\top G S$ is singular when both signs of a coefficient are allowed, but (3) is not: for $\lambda > 0$ the proof keeps the two halves of a coefficient out of $\mathcal{F}$ together, and the columns of X on the support are independent in general position [17]. The halves come apart only at $\lambda = 0$, which rounding can move to $\gamma \approx 10^{-16}$; `cvxcla` treats an event whose parameter is round-off as one at $\gamma = 0$.

The start Without equality rows the starting vertex is $z = 0$: all positions are closed, because each costs more than it can earn. The first event is the first entry, at $\gamma = \max_j |c_j|$ over the allowed signs, which is LARS's $\lambda_{\max}$. With equality rows the start is the linear program $\min \mathbf{1}^\top z$ over $ASz = b$, $0 \le z \le \bar{u}$, the point of smallest ℓ_1 norm on the constraint set. For generic A and $b \neq 0$ it is a unique vertex with m nonzeros. The path then holds still on its first segment, $\delta = 0$, while the multipliers absorb the growing penalty, until a second coefficient enters.

Figure 2(a) shows the penalty form on the diabetes data [4], where there are no constraints and LARS can check every knot.

4 The budget form

The penalty form cannot take a zero sum directly: with $A\beta = 0$ its start is degenerate, and it reaches one only by relaxing the constraint (Section 6). The second encoding starts instead from Tibshirani's original statement, the budget $\|\beta\|_1 \le t$, and divides by t. The split coefficients then add up to at most one, which is Markowitz's own setting: a long-only portfolio that invests a budget of one. A cash asset, with no risk and no return, holds whatever part of the budget the regression does not use.

The bound form has the same path as L_λ, indexed by $t = \|\beta(\lambda)\|_1$ instead of λ [17].

Proposition 2 (Budget as simplex)**.** *Suppose $b = 0$ and $l_j, u_j \in \{0, \pm\infty\}$, so that every constraint is homogeneous. Write $\beta = tSw$ with $w \ge 0$, and add a riskless asset $w_0 \ge 0$ with $\mathbf{1}^\top w + w_0 = 1$. Then for $t > 0$ the bound-form LASSO is the CLA instance*

$$\Sigma = \begin{bmatrix} S^\top G S & 0 \\ 0 & 0 \end{bmatrix}, \quad \mu = \begin{bmatrix} S^\top c \\ 0 \end{bmatrix}, \quad q = 0, \quad \gamma = 1/t,$$

with rows $\mathbf{1}^\top w + w_0 = 1$ and $ASw = 0$ and bounds $w, w_0 \ge 0$. This is Markowitz's long-only, fully invested problem with a cash asset of zero variance and zero return. If η is the multiplier of the budget row and ν those of $ASw = 0$, then $\lambda = t\eta$ and the LASSO's multipliers for $A\beta = 0$ are $t\nu$.

Proof. As in Proposition 1, $\|\beta\|_1 \le t$ is $\mathbf{1}^\top z \le t$ for the split $z = tw$, and w_0 is its slack. The constraint $ASz = 0$ is unchanged by the scale t, which is where homogeneity is needed. A nonzero right-hand side would become $b/t = \gamma b$, a constraint that moves with the parameter. Substituting $z = tw$ gives the objective $t^2(\frac{1}{2} w^\top S^\top G S w - \gamma c^\top S w)$ plus a constant, and dividing by $t^2 > 0$ gives P_γ. For the multipliers, multiply the CLA's stationarity row for a free coordinate, $(S^\top G S w)_j - \gamma (S^\top c)_j + \eta + (S^\top A^\top \nu)_j = 0$, by t. This gives $(S^\top (G\beta - c))_j + t\eta + (S^\top A^\top t\nu)_j = 0$, the LASSO's stationarity condition with penalty $t\eta$ and constraint multipliers $t\nu$. The cash row reads $\eta \ge 0$ while cash is held at zero. □

In code, cash is one more asset with no variance and no return, and the run stops when it enters:

```
k = 2 * p + 1                                   # long, short, cash
Sig = np.zeros((k, k)); Sig[:-1, :-1] = S.T @ G @ S
mu = np.append(S.T @ c, 0.0)
rows = np.vstack([np.ones(k), np.hstack([A @ S, np.zeros((m, 1))])])
run = CLA(mean=mu, covariance=Sig, a=rows, b=np.append(1.0, np.zeros(m)),
          lower_bounds=np.zeros(k), upper_bounds=np.full(k, np.inf), stop=k - 1)
knots = [(t.lamb, S @ t.weights[:-1] / t.lamb)   # beta = t S w with t = 1/gamma
         for t in run.turning_points]
```

The two ends of the CLA path have plain meanings. At $\gamma = \infty$, $t = 0$, the start is the linear program $\max c^\top S w$ over the simplex: put the whole budget on the best asset. Without

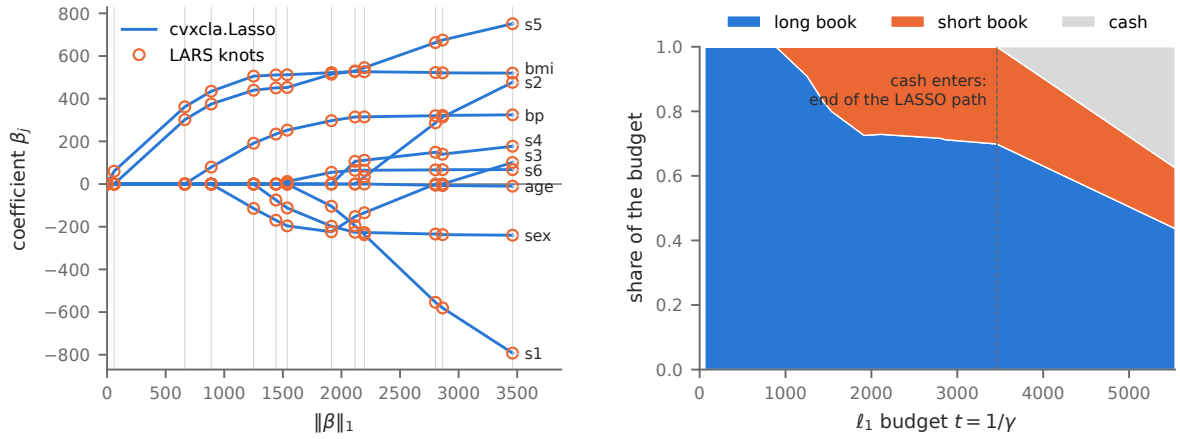


Figure 2: The diabetes LASSO path [4], both ways. *(a)* The CLA in the penalty form (lines) and the knots `lars_path` returns [15] (circles); all 13 agree to 6.9×10^{-14}, relative. *(b)* The same path seen by the budget form as a portfolio: the long and short positions Sw and the cash weight w_0 share a budget of one, against $t = 1/\gamma$. Cash enters at $t = \|\beta(0)\|_1 = 3460$, the end of the LASSO path; to its right the portfolio is the least-squares fit diluted by cash. The shares are read off the path: at budget t the books hold $\sum_j \beta_j^{\pm}/t$.

equality rows that is the predictor most correlated with the response, entering with the sign of its correlation and with $\lambda = \lim \eta/\gamma = \max_j |c_j|$, just as in LARS. Under $\mathbf{1}^\top \beta = 0$ it is one long and one short position of weight $\frac{1}{2}$ each, the most and the least correlated predictors, entering together. That is why two coefficients leave zero at once in Figure 3. Both starts are generically unique and nondegenerate.

At the other end, cash leaves its bound when its gradient η reaches zero, so $\lambda = 0$ and the budget goes slack: the regression no longer wants a larger ℓ_1 norm, and the LASSO path is over. Past that point the CLA still has a path, the ray $w = \beta(0)/t$ diluted by cash, but the twin halves' gradients vanish identically along it. The run therefore ends when cash enters, which `cvxcla`'s `stop` does, and that event wins a tie. A tie would arise when $p > n$, where every correlation vanishes at $\lambda = 0$ together with η and rounding alone would decide which event the walk takes. There the path ends where the fit first interpolates, the limit $\lambda \to 0^+$, and `cvxcla` stops at it (Section 6). Figure 2(b) shows the diabetes path in these terms.

The two forms give the same knots where $t = \|\beta(\lambda)\|_1$ is strictly decreasing in λ [17, Theorem 1 and Corollary 1], which fails only if the support's sign vector lies in the span of the active constraint normals. Under $\mathbf{1}^\top \beta = 0$ that would need every nonzero coefficient to have the same sign, impossible for a nonzero β that sums to zero.

Algorithm 1: the constrained LASSO path from a CLA.

1. Form $G = X^\top X$, $c = X^\top y$ and the split S of Section 3.
2. If there are equality rows and every constraint is homogeneous ($b = 0$, $l_j, u_j \in \{0, \pm\infty\}$, no inequality rows), use the budget form of Proposition 2. Run the CLA from $\gamma = \infty$ and stop when cash leaves zero. A turning point at γ with budget multiplier η is the knot $\beta = Sw/\gamma$ at $\lambda = \eta/\gamma$.
3. Otherwise use the penalty form of Proposition 1 and run the CLA from $\gamma = \infty$ to 0. A turning point at γ is the knot $\beta = Sz$ at $\lambda = \gamma$. With $b \neq 0$, or a row with $h_r < 0$, the start is the feasible point of smallest ℓ_1 norm (Section 6).
4. Between two knots β is affine in λ, so any other point on the path is an interpolation.

`cvxcla.Lasso(X, y, a=A, b=b, g=C, h=h, lower=l, upper=u)` (release 2.7.0) carries out steps 1–4 and returns the knots. With no constraint at all it traces the plain LASSO by a homotopy instead.

5 Numerical evidence

The checks test predictions of [17]. Section 2 and Lemma 1 there cast L_λ and the CLA as instances of one parametric quadratic program, so the penalty form should return the knots of L_λ. Theorem 1 and Corollary 1 there say that the capped program, which the budget form solves, has the same knots wherever $\|\beta(\lambda)\|_1$ is strictly decreasing, so the budget form should return them too. The author of `cvxcla` wrote the checks, so they are not independent in the usual sense. The references are: LARS, an interior-point solver, the optimality conditions evaluated without either encoding, and three path algorithms written by others. The scripts and their tolerances are published.

Table 2 lists the test problems. The diabetes problems use the data of Section 1, with the response centred. *Nonnegative* requires every coefficient to be at least zero, the one constraint `lars_path` accepts. *Wide* is a Gaussian design with $n = 40$ and $p = 100$, with 8 nonzero coefficients and noise of standard deviation 0.5. *Box* is the opening example. At $\lambda = 0$ the variables `bmi` and `s5` sit at the cap, while `s1`, whose least-squares coefficient is the largest, ends well inside it. *Cholesterol* caps instead the net effect of the three cholesterol measures, $|\beta_{\mathtt{s1}} + \beta_{\mathtt{s2}} + \beta_{\mathtt{s3}}| \leq h$, two inequality rows with h half the least-squares value; the rows bind at the end of the path. *Pinned* requires the fitted value for the first patient to equal the observed one, one nonhomogeneous equality row. Its starting vertex puts all weight on `bmi`, the largest entry of that patient's covariate vector, and holds still until $\lambda = 4053$. *Zero-sum* is a regression on compositional data in the style of Lin et al. [10]. Each of $n = 60$ samples is a composition of $p = 30$ parts, the predictors are the logs of the shares, and the coefficients must sum to zero: a composition is only known up to scale, and only a zero sum makes the fit independent of the units. Five parts have nonzero true coefficients. *Zero-sum, signs* adds nonnegativity on the even-numbered parts, which the truth meets, and *Zero-sum, caps* a cap at half the largest zero-sum least-squares coefficient. *Monotone* is the ordered fit of Section 6, with rows $\beta_j \leq \beta_{j+1}$. *Sum to one* predicts one unit's outcomes over 40 periods from those of 20 donors, as a synthetic control does, with weights that must sum to one and may be negative. *sCD14* is the same model on real data, from a study of the gut microbiome in HIV infection [12], as distributed with the R package `coda4microbiome` [3]: the counts of 60 bacterial genera in 151 stool samples, and as the response the level of sCD14, a marker of inflammation, in thousands. A count of zero has no logarithm, so each count gets a pseudocount of one, a common choice for such data; the predictors are then the logs of the shares, centred. Its path has 60 knots (Figure 3).

Every path comes from `cvxcla.Lasso` 2.7.0, with the bounds passed even where they are

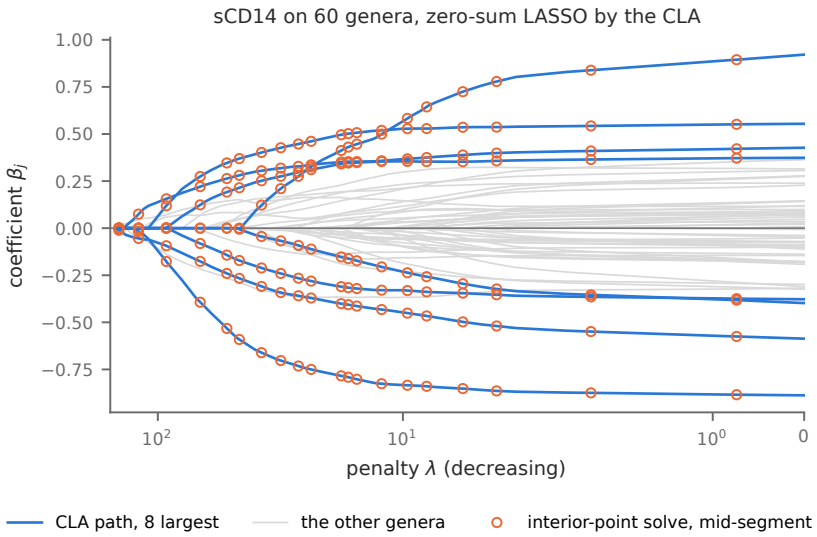


Figure 3: The zero-sum LASSO of sCD14 on the log-shares of 60 genera [12]: the path by the CLA in the budget form, the eight genera largest at $\lambda = 0$ in colour and the others in grey, with an interior-point solve at every third segment midpoint for those eight (circles). Table 2 checks every midpoint of every coefficient.

infinite, so that the CLA computes it. Where LARS applies it is the reference, compared knot by knot in both λ and β. Elsewhere the reference is an interior-point solve (Clarabel through CVXPY, tolerances 10^{-12}) at the midpoint of every segment, where the path is affine in λ and so known exactly from its two ends. This comparison is only as sharp as the solver: its largest gap in Table 2 is 4.8×10^{-8}, relative, where the optimality conditions of the same path hold to rounding. The third check evaluates the optimality conditions of (1) directly at every knot and along every segment. A linear program picks the multipliers κ of $A\beta = b$ and $\eta \geq 0$ of the active rows of $C\beta \leq h$ that satisfy the conditions best, and the violation is then evaluated exactly at those multipliers, so the solver's own tolerance cannot hide one. With $r = G\beta - c + A^\top\kappa + C^\top\eta$ the conditions are $r_j = -\lambda \operatorname{sign}\beta_j$ on the interior of the support; $|r_j| \leq \lambda$ at zero, or only $r_j \geq -\lambda$ under $\beta_j \geq 0$ and $r_j \leq \lambda$ under $\beta_j \leq 0$; $r_j \leq -\lambda$ at a cap $u_j > 0$ and $r_j \geq \lambda$ at a floor $l_j < 0$; $\eta \geq 0$; and primal feasibility. The check covers each segment whole, not only its midpoint, as Hobson et al. [6] do. Which coefficients are zero, at a bound or free, and which rows are active, is read off the midpoint. With that fixed, every condition is affine in λ once the multipliers are taken affine between the two ends, so it holds along the whole segment if it holds at both ends. A linear program finds multipliers at both ends under the midpoint's pattern, and both ends must fit that pattern: no coefficient may change sign or leave a bound, and no row may come loose, inside a segment. A path that is right at both ends of a segment and at its midpoint, but cuts across a knot, fails this check; the published tests include one. The largest violation, relative to $\|c\|_\infty$, is the last column of the table.

The CLA returns LARS's knots to 10^{-13}, including on the wide design, where 53 knots carry the path to the interpolating fit. The interior-point gaps are the solver's: the KKT residuals of the same paths are at the level of rounding throughout, including the problems LARS cannot take. In positive mode the last point `lars_path` returns is labelled $\alpha = 0$ but is the path at $\lambda = 30.65$, where it found nothing left to enter; its KKT violation at $\lambda = 0$ is 3.2×10^{-2}. The CLA carries on to the nonnegative least-squares fit, so the table compares that last knot with `scipy.optimize.nnls` [9] instead. On the real data the CLA is as accurate as on the simulated problems.

problem	n	p	knots	ref.	gap	KKT
plain	442	10	13	LARS	6.9×10^{-14}	1.1×10^{-15}
nonnegative	442	10	6	LARS	2.5×10^{-15}	1.1×10^{-15}
wide	40	100	53	LARS	4.2×10^{-13}	6.1×10^{-16}
box	442	10	13	IPM	4.8×10^{-8}	1.2×10^{-15}
cholesterol	442	10	14	IPM	3.4×10^{-9}	1.1×10^{-15}
pinned	442	10	18	IPM	3.6×10^{-10}	1.6×10^{-15}
zero-sum	60	30	30	IPM	1.1×10^{-10}	1.0×10^{-15}
zero-sum, signs	60	30	21	IPM	2.1×10^{-11}	1.0×10^{-15}
zero-sum, caps	60	30	35	IPM	1.4×10^{-9}	5.3×10^{-16}
monotone	100	10	13	IPM	4.6×10^{-12}	9.5×10^{-16}
sum to one	40	20	19	IPM	1.3×10^{-9}	7.5×10^{-16}
sCD14	151	60	60	IPM	8.8×10^{-10}	3.1×10^{-15}

Table 2: The CLA against LARS (knot by knot) and against an interior-point solver (IPM, at segment midpoints). *Gap*: largest discrepancy, relative to the largest coefficient. *KKT*: largest violation of the optimality conditions of (1) at every knot and along every segment, relative to $\|c\|_\infty$.

Scale Table 3 extends the test to simulated zero-sum problems of up to $p = 500$ parts. At that size a midpoint solve per segment would cost more than the path itself, so the table reports only the optimality conditions; they hold to rounding. A second check at the largest size comes from the other encoding: the penalty form, relaxed, returns the same 542 knots to 8.5×10^{-15} (Appendix B). For a sense of cost, a grid of 100 interior-point solves (Clarabel through CVXPY, compiled once with λ as a parameter) costs about 100 times as much as the whole CLA path at $p = 100$ and 350 times at $p = 500$, and 14 to 100 times on the constrained diabetes and zero-sum problems of Table 2. These are ratios of the fastest of several runs, measured once by `timings` on an Apple M4 Pro and quoted as measured; they are the one set of numbers here that the script does not write into the text, because they change from machine to machine. The grid also misses every knot between grid points.

Hostile problems The problems so far are easy for any path algorithm. Table 4 adds problems chosen to make one fail, each drawn with three seeds, and reports the worst case. *Correlated* is a Gaussian design with $n = 100$ and $p = 40$ in which every pair of columns has correlation 0.99999, alone, under a box and under a zero sum. *Duplicate* adds to the diabetes data an eleventh column, `bmi` plus noise of 1.0×10^{-4} or 1.0×10^{-6} times its spread, alone and under the cap of *Box*, so that two columns nearly coincide. *Wide, zero-sum* puts a zero sum on *Wide*, where $p > n$.

Every path that is traced meets the optimality conditions to 1.6×10^{-12} or better. The gaps to the references grow with the conditioning, and the references account for them. Against LARS the gap stays below $\mathrm{cond}(X)^2$ times the machine epsilon, the scale at which conditioning lets two exact solvers part, and LARS's own KKT residual is at most 1.1×10^{-13}: both paths are right. Against the interior-point solver the gap reaches 2.5×10^{-5} on *Duplicate*, but the solver's objective is never lower than the path's by more than 4.8×10^{-13}, relative, which is within its tolerance. That gap is the solver's.

`cvxcla` declines one of the problems, for every seed. Without a cap, *Duplicate* at 1.0×10^{-6} brings both twins into the free set, where the bordered system is singular to working precision, and `cvxcla` stops with an error. Under the cap the duplicate enters first and reaches the cap before `bmi` enters, so the twins are never free together, and the path is exact. *Wide, zero-sum*, where $X^\top X$ is singular, is traced, and with $\mathrm{cond}(X)$ near 10^{16} it has the largest residual in the table, 1.6×10^{-12} (Section 6).

More predictors than observations *Wide, zero-sum* is the only problem so far with both $p > n$ and a constraint, and a zero sum alone has an exact path in comlasso [8] too. Table 5 puts on *Wide*, again with $n = 40$, $p = 100$ and three seeds, the constraints comlasso does not take: nonnegativity; a box at half the largest coefficient of the least-squares fit of smallest norm; two inequality rows that cap the net effect of three predictors, as in *Cholesterol*, and bind; the row of *Pinned*; and a zero sum with the signs of *Zero-sum, signs* or with the caps of the box. `cvxcla` declines none of them. Every path meets the optimality conditions to 3.2×10^{-13} or better along every segment, and the interior-point solver agrees to 8.7×10^{-7}. Its objective is never lower than the path's by more than 4.9×10^{-12}, relative, and that only near the end of a path, where the fit almost interpolates and the objective is small.

Three independent path algorithms Three path algorithms derived for the LASSO, with no reference to portfolios, can be run against the CLA. All three trace its path, up to the ridge two of them add.

comlasso [8] traces the exact path under zero sums on blocks of coefficients, with no ridge and with $p > n$, so it should agree with the CLA with no workaround. Its authors publish it as an R package built for R 3.2, which we load unchanged (Code availability) and run on *Zero-sum*, *sCD14* and *Wide, zero-sum*. It returns the CLA's knots, to 4.3×10^{-15} relative to the largest coefficient on the first two and to 6.5×10^{-12} on *Wide, zero-sum* over the three seeds. On two of those seeds it takes one more step, to a λ at rounding level, before it records $\lambda = 0$; `cvxcla` counts such an event as $\lambda = 0$ (Section 3).

The algorithm of Gaines et al. [5] ships in MATLAB and in Julia. We ported the Julia package to Julia 1 without changing the algorithm (Code availability) and ran it on every problem of Table 2. Started at the exact first knot, where that is simple to state, it agrees with the CLA to 2.9×10^{-14} on the five such problems with $n \geq p$. On *Wide*, where $n < p$, its ridge keeps it 2.8×10^{-2} away. As shipped, its own start puts it off the path on all 12 problems, by 1.6×10^{-5} to 1.1×10^{2} relative to the largest coefficient; Appendix C says why.

`c-lasso` [20] has a path algorithm for $C\beta = 0$ alone, without bounds or signs, so it runs on *Zero-sum*, *sCD14* and *Wide, zero-sum*. It adds a ridge term $10^{-3}I$ to $X^\top X$, fixed in the package, and that term is the whole of its distance to the CLA: on the first two its 100 grid points lie up to 6.1×10^{-5} from the CLA path as shipped, relative to the largest coefficient, and 6.0×10^{-14} with the ridge set to 10^{-12} by hand. With $p > n$ the same ridge is not small. On *Wide, zero-sum*, with its grid carried down to 1.0×10^{-9} times its largest λ so that it reaches the end of the path, `c-lasso` as shipped lies 0.41 to 0.58 times the largest coefficient from the CLA over the three seeds, and 4.9×10^{-12} with the ridge at 10^{-12}.

Two further packages fit on a grid of λ rather than tracing the path, so they cannot check the knots and are not run: `zeroSum` [1], for zero-sum elastic nets in R, and `PACLasso` [14], the penalized and constrained LASSO of James et al. [7], which CRAN archived on 2 June 2025.

6 Where the two forms stop

Alone, the two forms cover bounds, equality rows with a nondegenerate start, inequality rows that are slack at $\beta = 0$ ($h > 0$) or exclude it ($h < 0$), and homogeneous rows with signs. They stop in two places: at a degenerate start, and at a free block that is singular to working precision. This section says where each arises, what `cvxcla` does about it, and what stays out of reach.

Degenerate starts With $b = 0$ the penalty form's starting vertex is $z = 0$, with no free coordinates for $m > 0$ rows. This is a degenerate vertex: the multipliers at the start are not determined by the active set, and (3) is singular. For homogeneous equality rows with signs the budget form is the remedy, because rescaling to the simplex moves the start off the origin. It cannot take rows or caps: rescaled by t, a row $C\beta \leq h$ becomes $CSw \leq \gamma h$ and a cap $\beta_j \leq u_j$

n	p	knots	KKT
200	100	100	1.1×10^{-15}
400	200	212	1.4×10^{-15}
1000	500	542	1.9×10^{-15}

Table 3: The budget form on simulated zero-sum problems of growing size: the largest KKT violation at every knot and along every segment, as in Table 2.

problem	n	p	cond(X)	knots	ref.	gap	KKT
correlated	100	40	5.5×10^{3}	43–45	LARS	1.8×10^{-9}	9.8×10^{-14}
correlated, box	100	40	5.5×10^{3}	47–54	IPM	6.5×10^{-8}	7.3×10^{-14}
correlated, zero-sum	100	40	5.5×10^{3}	42	IPM	4.3×10^{-8}	3.8×10^{-13}
duplicate 10^{-4}	442	11	3.2×10^{4}	14	LARS	2.4×10^{-7}	2.9×10^{-13}
duplicate 10^{-4}, box	442	11	3.2×10^{4}	14	IPM	2.1×10^{-5}	1.1×10^{-15}
duplicate 10^{-6}	442	11	3.2×10^{6}			declined	
duplicate 10^{-6}, box	442	11	3.2×10^{6}	14	IPM	2.5×10^{-5}	1.6×10^{-15}
wide, zero-sum	40	100	5.3×10^{15}	50–78	IPM	1.5×10^{-8}	1.6×10^{-12}

Table 4: Hostile problems, through the checks of Table 2, the worst over three seeds. *Declined*: `cvxcla` stops with an error. The duplicate column is `bmi` plus noise of the given size, relative to its spread.

becomes $w \le \gamma u_j$, right-hand sides that move with the parameter. So inequality rows or finite bounds under a zero sum leave only the penalty form, which starts degenerate there. And homogeneous rows ($h = 0$), such as the ordering constraints of a monotone fit, are all tight at $\beta = 0$, so the start is degenerate in either form.

`cvxcla` resolves both by relaxation. It traces the problem twice, with the homogeneous rows relaxed to $h_r = \varepsilon$ and $h_r = 2\varepsilon$, and a zero sum to $A\beta = \varepsilon Ad$ and $A\beta = 2\varepsilon Ad$ for a point d the constraints admit. Both runs start nondegenerate, and Lemma 1 combines them.

Lemma 1. *Suppose the penalty form, relaxed by ε and by 2ε, passes through the same sequence of free sets, blocked sets and active rows. Then each knot (λ_k, β_k) is affine in the relaxation, and $2(\lambda_k, \beta_k)(\varepsilon) - (\lambda_k, \beta_k)(2\varepsilon)$ is its value at relaxation zero.*

Proof. The relaxation enters (3) only through the part of the right-hand side that does not multiply γ, by way of b and of h on the active rows; it never enters the matrix or the term in γ. On a segment, therefore, α, ν_0, g_0 and the rows' slacks at $\gamma = 0$ are affine in the relaxation, while δ, ν_1, g_1 and the slacks' slopes do not depend on it. Every event is $\gamma = -(\text{intercept})/(\text{slope})$ with an affine intercept and a fixed slope, so the knot $\lambda_k = \gamma_k$ is affine in the relaxation, and so is $x_k = \alpha + \gamma_k \delta$. A coordinate blocked at the knot sits at a bound, which does not depend on the relaxation, so the next segment's right-hand side is again affine in it, and induction over the segments covers the path. An affine f has $f(0) = 2f(\varepsilon) - f(2\varepsilon)$. □

At relaxation zero some segments have length zero: the relaxed paths meet rows one by one that the exact path meets together. Those are dropped. The lemma's hypothesis cannot be checked in advance, and its conclusion is a candidate path, not a proof that it is the LASSO's. So the result is certified, not trusted: a linear program finds the multipliers that best satisfy the optimality conditions of (1) at every knot and segment midpoint, and the path is returned only if the largest violation, relative to $\|c\|_\infty$, is at most 10^{-9}.

The relaxation is relative to the size of a coefficient, $\varepsilon = \rho \, \|c\|_\infty / \max_j G_{jj}$, with $\rho = 10^{-6}$ first. The point d gives each coefficient a magnitude drawn uniformly from $[\frac{1}{2}, \frac{3}{2}]$, from a fixed

Wide with	knots	gap	KKT
nonnegative	56–100	2.2×10^{-9}	2.2×10^{-15}
box	92–228	2.8×10^{-8}	1.3×10^{-15}
inequality rows	54–62	8.0×10^{-8}	1.1×10^{-14}
pinned	61–73	3.0×10^{-8}	1.4×10^{-15}
zero-sum, signs	44–56	4.7×10^{-9}	3.2×10^{-13}
zero-sum, caps	88–245	8.7×10^{-7}	1.6×10^{-13}

Table 5: More predictors than observations, $n = 40$ and $p = 100$, under constraints beyond a zero sum, through the checks of Table 2 against the interior-point solver, the worst over three seeds. None is declined.

seed, and a sign its bounds allow, so that Ad is generic and the relaxed start a unique vertex; d is then scaled so that $C_r d \le \frac{1}{2}$ on the homogeneous rows, which keeps εd strictly inside their relaxation. If the two runs trace different numbers of knots, or the certificate fails, `cvxcla` tries $\rho = 10^{-7}$ and then 10^{-8}; below about 10^{-9} the relaxed start is lost in rounding. It declines the problem when none passes. *Monotone* and *Zero-sum, caps* test the relaxation (Table 2). Appendix A shows on the monotone fit that the knots move linearly in ε, as the extrapolation needs, and Appendix B that the relaxed penalty form reproduces the budget form on every zero sum.

Starts that need no relaxation Rows $C\beta \le h$ with $h > 0$ enter the penalty form as rows $CSz \le h$ of the CLA. They are slack at the start $z = 0$, so the start is unchanged, and a row joins the active set when the path reaches it, like a bound. *Cholesterol* tests this (Table 2). Two other starts are nondegenerate but lie away from $\beta = 0$. A nonhomogeneous row can make the starting linear program non-unique. The fully invested constraint $\mathbf{1}^\top \beta = 1$ of sparse portfolio regression [2] is the standard case: every long-only β in the simplex has $\|\beta\|_1 = 1$ and minimises the ℓ_1 norm. The path's first segment is then the least-squares fit over that face, constant in λ. This is the flat segment of [17, Proposition 2], and finding the point requires a quadratic program rather than a linear one. `cvxcla` starts on the tied face where its quadratic term is smallest, which is that fit, so the walk begins on the flat segment. *Sum to one* tests it (Table 2). A row with $h_r < 0$ leaves $\beta = 0$ infeasible. The budget form starts at $\beta = 0$ and cannot take it. The penalty form starts where the linear program of Section 2 puts it, at the feasible point of smallest ℓ_1 norm, as it does for a nonhomogeneous equality row (*Pinned*); Gaines et al. and CLASSO [6] also start from a feasible point. *Floor* tests this: the diabetes LASSO with $\beta_{\texttt{bmi}}$ at least half its least-squares value. Its path starts on the floor, with `bmi` alone, has 13 knots, lies within 1.1×10^{-10} of an interior-point solve at every midpoint and meets the optimality conditions to 1.0×10^{-15}.

More predictors than observations With equality rows and $p > n$, $X^\top X$ is singular and the end of the path, the constrained least-squares fit, need not be unique. Before it, the free set can outgrow the rank of X, and the bordered system is then solved on the null space of the active constraints, which keep it nonsingular. The path ends where the fit first interpolates, the limit $\lambda \to 0^+$: the interpolating fit of smallest ℓ_1 norm under the constraints. Past it the CLA would run on through interpolating fits of larger norm, which are not on the LASSO path, so `cvxcla` stops there. *Wide, zero-sum* (Table 4) tests this, and Table 5 tests it under bounds, signs and inequality rows. The other path algorithms for general constraints do not give this path. Gaines et al. add a ridge of 10^{-4} when $n < p$, `c-lasso` always adds one of 10^{-3}, and CLASSO [6], not run here, adds one of $10^{-4} \max_j G_{jj}$. Each then traces a nearby problem whose path ends elsewhere. On *Wide, zero-sum* the ridge of `c-lasso` moves its path by up to

0.58 times the largest coefficient (Section 5). On their own problems with $p > n$, Hobson et al. report objectives up to 4×10^{-2}, relative, above the unregularised ones at their default ridge [6, Table 3]. For a zero sum alone, comlasso [8] traces the exact path with $p > n$ and no ridge, and its authors make the same case against one; on *Wide, zero-sum* it agrees with the CLA to 6.5×10^{-12} (Section 5).

What stays out of reach Two things stop the run with an error rather than a wrong path. The bordered system (3) needs the columns on the free set to be independent; when two columns nearly coincide and both become free, it is singular to working precision and `cvxcla` stops (*Duplicate* at 1.0×10^{-6}, Table 4). This is the general-position assumption of Section 1, failing in floating point. And a relaxation can fail its certificate, when knots of the exact path lie closer together than the relaxation moves them; `cvxcla` then retries with smaller relaxations and declines the problem if none passes. Among the problems here only the largest zero sum needed a retry (Appendix B), and none was declined. Bounds that exclude zero are not a limit: they enter as rows.

7 Conclusion

Schmelzer and Hastie [17] prove that a constrained LASSO is a portfolio problem and that its path is the path of the Critical Line Algorithm (their Lemma 1, Theorem 1 and Corollary 1). The checks here support the identity wherever the encodings reach. On every problem, a CLA written for portfolios either returns the exact knots or stops with an error when its arithmetic cannot be trusted. Where an encoding starts degenerate, `cvxcla` relaxes the constraint, extrapolates and certifies the result, so the checks reach monotone fits and caps and rows under a zero sum. Two other starts need no relaxation: weights that sum to one start on a flat segment, and rows that exclude $\beta = 0$ start at the feasible point of smallest ℓ_1 norm. With $p > n$ the paths stay exact, with no ridge, under bounds, signs, inequality rows and zero sums, where the other path algorithms for general constraints trace a ridged problem that ends elsewhere. The checks also made the CLA better at portfolios, where it now takes tied forecasts and low-rank covariances. The practical consequence is simple: `lars_path` cannot take even a cap on the coefficients, and a CLA that takes general linear rows can.

Code and data availability

The software is in the directory `preprints/constrained_lasso/` of https://github.com/Jebel-Quant/lasso, under the MIT licence. Each script declares its pinned dependencies and has a lockfile next to it that fixes Python 3.12 and every package by version and hash. `python/make_figures.py` computes every path with `cvxcla.Lasso` 2.7.0 [18], holds every check, and writes every figure, table and number the text quotes; its `test` mode runs the checks and `timings` the cost ratios. `python/classo_check.py` runs `c-lasso` 1.0.11 under NumPy 1.26. `R/comlasso_check.R` runs comlasso 0.2, fetched from its authors' repository https://github.com/jenjong/ComLasso at a pinned commit and checked by SHA-256, with `Rglpk` from CRAN; R 4 will not install a package built before R 4.0, so the script loads comlasso's code from the package unchanged. `julia/classopath.jl` is the port of the path algorithm of `ConstrainedLasso.jl` (MIT licence) to Julia 1.12, with each departure from the original marked in it: the sign convention of the solver's multipliers, Julia 0.6's handling of NaN in `findmin`, and a stray character; none changes the algorithm. `julia/gaines_check.jl` runs it, in the environment its `Manifest.toml` pins. The simulated problems use fixed seeds. The sCD14 data are not redistributed: `python/fetch_scd14.py` downloads `coda4microbiome` 0.2.4 from CRAN and checks its SHA-256. `make figures` runs all five, in CI as well.

The lockfiles fix the software, not the hardware. Residuals near rounding come out with different last digits on another CPU or BLAS build, so a rerun will not match the printed values digit for digit. The tests therefore check the optimality conditions against tolerances between 10^{-12} and 10^{-10}, and the gaps to the interior-point solver against its own accuracy. Costs are given only as ratios, which vary far less between machines than absolute times.

Competing interests

The author works at Jebel Quant Research, which hosts the code, and maintains `cvxcla`, with which every path is computed. Every path is checked without it: against LARS, an interior-point solver and the LASSO's optimality conditions. There are no other competing interests.

Acknowledgements

I thank Trevor Hastie for many helpful discussions.

References

[1] M. Altenbuchinger, T. Rehberg, H. U. Zacharias, et al. Reference point insensitive molecular data analysis. *Bioinformatics*, 33(2):219–226, 2017. doi:10.1093/bioinformatics/btw598.

[2] J. Brodie, I. Daubechies, C. De Mol, D. Giannone, and I. Loris. Sparse and stable Markowitz portfolios. *Proceedings of the National Academy of Sciences*, 106(30):12267–12272, 2009. doi:10.1073/pnas.0904287106.

[3] M. L. Calle, M. Pujolassos, and A. Susín. coda4microbiome: compositional data analysis for microbiome cross-sectional and longitudinal studies. *BMC Bioinformatics*, 24:82, 2023. doi:10.1186/s12859-023-05205-3.

[4] B. Efron, T. Hastie, I. Johnstone, and R. Tibshirani. Least angle regression. *The Annals of Statistics*, 32(2):407–499, 2004. doi:10.1214/009053604000000067.

[5] B. R. Gaines, J. Kim, and H. Zhou. Algorithms for fitting the constrained lasso. *Journal of Computational and Graphical Statistics*, 27(4):861–871, 2018. doi:10.1080/10618600.2018.1473777.

[6] M. Hobson, L. Butcher, and L. Lentati. CLASSO: a practical primal path-following algorithm for the constrained LASSO. arXiv preprint arXiv:2610.17024, 2026. doi:10.48550/arXiv.2610.17024.

[7] G. M. James, C. Paulson, and P. Rusmevichientong. Penalized and constrained optimization: an application to high-dimensional website advertising. *Journal of the American Statistical Association*, 115(529):107–122, 2020. doi:10.1080/01621459.2019.1609970.

[8] J.-J. Jeon, Y. Kim, S. Won, and H. Choi. Primal path algorithm for compositional data analysis. *Computational Statistics & Data Analysis*, 148:106958, 2020. doi:10.1016/j.csda.2020.106958.

[9] C. L. Lawson and R. J. Hanson. *Solving Least Squares Problems*. Prentice-Hall, 1974.

[10] W. Lin, P. Shi, R. Feng, and H. Li. Variable selection in regression with compositional covariates. *Biometrika*, 101(4):785–797, 2014. doi:10.1093/biomet/asu031.

[11] H. M. Markowitz. The optimization of a quadratic function subject to linear constraints. *Naval Research Logistics Quarterly*, 3(1–2):111–133, 1956. doi:10.1002/nav.3800030110.

[12] M. Noguera-Julian, M. Rocafort, Y. Guillén, et al. Gut microbiota linked to sexual preference and HIV infection. *EBioMedicine*, 5:135–146, 2016. doi:10.1016/j.ebiom.2016.01.032.

[13] M. R. Osborne, B. Presnell, and B. A. Turlach. A new approach to variable selection in least squares problems. *IMA Journal of Numerical Analysis*, 20(3):389–403, 2000. doi:10.1093/imanum/20.3.389.

[14] C. Paulson, G. M. James, and P. Rusmevichientong. PACLasso: penalized and constrained lasso optimization. R package version 1.0.0, 2019; archived on CRAN on 2 June 2025. doi:10.32614/CRAN.package.PACLasso.

[15] F. Pedregosa et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[16] S. Rosset and J. Zhu. Piecewise linear regularized solution paths. *The Annals of Statistics*, 35(3):1012–1030, 2007. doi:10.1214/009053606000001370.

[17] T. Schmelzer and T. Hastie. The Critical Line Algorithm and the constrained LASSO: one curve, two literatures. arXiv preprint arXiv:2609.25704, 2026. doi:10.48550/arXiv.2609.25704.

[18] T. Schmelzer. cvxcla: the Critical Line Algorithm with an operator interface in Python. SSRN preprint 7583179, 2026. doi:10.2139/ssrn.7583179.

[19] T. Schmelzer. The efficient frontier from a LASSO solver: a numerical check of the CLA–LASSO identity. *Journal of Reproducible Statistics*, 1, 2026. doi:10.68365/jrepstat.2026.7.

[20] L. Simpson, P. L. Combettes, and C. L. Müller. c-lasso – a Python package for constrained sparse and robust regression and classification. *Journal of Open Source Software*, 6(57):2844, 2021. doi:10.21105/joss.02844.

[21] R. Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B*, 58(1):267–288, 1996. doi:10.1111/j.2517-6161.1996.tb02080.x.

[22] R. J. Tibshirani and J. Taylor. The solution path of the generalized lasso. *The Annals of Statistics*, 39(3):1335–1371, 2011. doi:10.1214/11-AOS878.

[23] R. J. Tibshirani. The lasso problem and uniqueness. *Electronic Journal of Statistics*, 7:1456–1490, 2013. doi:10.1214/13-EJS815.

[24] P. W. F. Wilson, J. B. Meigs, L. Sullivan, et al. Prediction of incident diabetes mellitus in middle-aged adults: the Framingham Offspring Study. *Archives of Internal Medicine*, 167(10):1068–1074, 2007. doi:10.1001/archinte.167.10.1068.

[25] H. Zhou and K. Lange. A path algorithm for constrained estimation. *Journal of Computational and Graphical Statistics*, 22(2):261–283, 2013. doi:10.1080/10618600.2012.681248.

A The extrapolation on the monotone fit

Lemma 1 needs the relaxed paths to keep their sequence of active sets as the relaxation shrinks. The monotone fit illustrates that they do; it is an example, not a proof. On a Gaussian design with $n = 100$, $p = 10$ and an ordered truth, the rows are $\beta_j - \beta_{j+1} \le h$. For every ε from 1.0×10^{-3} down to 1.0×10^{-7} the path at $h = \varepsilon$ has the same 16 knots, meets its own optimality conditions to 9.7×10^{-16}, and lies ε from the path at $h = 0$, to two digits: the knots move linearly in ε, as the extrapolation needs. Some of its segments shrink to a point at $h = 0$, where the exact path has 13 knots.

B One encoding or two

With the relaxation, the penalty form takes a zero sum too, which raises the question whether the budget form is still needed. Table 6 sends every zero-sum problem of Tables 2 to 4 through the penalty form as well, with caps so loose they never bind. On every one it returns the budget form's knots to rounding. The largest, $p = 500$ parts and over 500 knots, was declined by `cvxcla` 2.6.0, which tried a single relaxation. Its knots crowd together, the closest two 2.4×10^{-8} times $\lambda_{\max}$ apart, and where two knots lie closer than the relaxation moves them, the two relaxed runs can meet them in different orders. The extrapolation then pairs knots of different events, and the certificate rejects the result. Release 2.7.0 retries with smaller relaxations until the two runs agree, and returns the path only once it passes the certificate. The budget form traces the same path in one run (Table 3), while the relaxation costs at least a second. So the penalty form is

the general encoding, and the budget form is the direct one for homogeneous constraints, and the one in which a regression is visibly a portfolio (Figure 2(b)).

problem	budget	penalty	gap	KKT
zero-sum	30	30	1.7×10^{-15}	7.8×10^{-16}
zero-sum, signs	21	21	1.1×10^{-15}	7.6×10^{-16}
sCD14	60	60	1.3×10^{-14}	3.3×10^{-15}
wide, zero-sum	50	50	2.4×10^{-13}	1.0×10^{-15}
zero-sum, $p = 500$	542	542	8.5×10^{-15}	2.1×10^{-15}

Table 6: Every zero-sum problem through both encodings: the knots of the budget form, and of the penalty form with the zero sum relaxed, extrapolated and certified. *Gap*: largest knot discrepancy between the two, relative, as in Table 2; *KKT*: of the penalty form's path. *Wide, zero-sum* is the first of the seeds of Table 4.

C The algorithm of Gaines et al.

The Julia package of Gaines et al. [5], `ConstrainedLasso.jl`, was written for Julia 0.6 and no longer loads. We ported it to Julia 1 and the current `Convex.jl` without changing the algorithm, and ran it with bounds as inequality rows. The five problems on which Section 5 starts it at the exact first knot are *Plain*, *Box*, *Cholesterol*, *Zero-sum* and *sCD14*. On *Nonnegative*, *Zero-sum, signs*, *Zero-sum, caps*, *Monotone*, *Pinned* and *Sum to one* the multipliers at the first knot are not unique, or the first knot is a face rather than a vertex, so we give it no start there and make no claim. As shipped, it takes the first knot from a linear program that does not involve y, which starts *Zero-sum* at $\lambda = 69.8$ instead of 63.4, and it calls a coefficient active there when an interior-point solve puts it above 10^{-4}, an absolute threshold that on the diabetes data, whose loss is of order 10^6, admits ten of the ten. The MATLAB original (`lsq_classopath.m`, lines 165–189, 242 and 253), which we did not run, reads its first λ off that start's multipliers instead.

D What the checks did for the CLA

Two of the cases in Section 6 were defects of the CLA, not of the encodings, and both refuse ordinary portfolio problems. A tie for the largest expected return, two assets with the same forecast, makes the starting vertex non-unique; the CLA of `cvxcla` 2.5.0 refused it, and 2.6.0 starts on the tied face where the quadratic term is smallest, the fix the flat start needed. A covariance of low rank, such as a factor model without specific risk, makes the free block singular; 2.5.0 refused it, and 2.6.0 solves it on the null space of the active constraints, the fix the case $p > n$ needed. Table 7 traces both with 2.7.0 and checks every turning point against an interior-point solve of the same Markowitz problem. Checking the identity on regressions thus made the portfolio solver better at portfolios.

portfolio problem	n	rank	turning points	worse
tied maximum return	8	8	8	$< 10^{-15}$
factor model, no specific risk	12	3	6	$< 10^{-15}$

Table 7: Two long-only, fully invested Markowitz problems that the CLA of `cvxcla` 2.5.0 refused, traced since 2.6.0. *Worse*: the most by which a turning point's objective exceeds an interior-point solve of the same problem, relative to the objective's size at the start; it is within the solver's tolerance.